

Projeto — Sistema de Combate de um RPG

Você está desenvolvendo o núcleo de um RPG baseado em turnos. O objetivo é construir um pequeno sistema de combate utilizando **programação orientada a objetos**, explorando classes, encapsulamento, associação, composição, interfaces e polimorfismo.

O projeto deve ser desenvolvido **sem utilizar herança entre os diferentes tipos de personagens, armas, habilidades ou efeitos**. Prefira composição e interfaces.

1. O jogo

Crie uma classe **Jogo** que represente o jogo em execução.

O jogo deve:

- manter os personagens que participam da partida;
- permitir adicionar personagens;
- controlar a passagem dos turnos;
- permitir que objetos que precisam reagir à passagem de um turno sejam registrados no jogo;
- ao passar um turno, notificar esses objetos.

Crie uma interface que represente objetos que precisam ser atualizados a cada turno:

```
TypeScript
interface AtualizavelPorTurno {
  novoTurno(): void;
}
```

O **Jogo** deve trabalhar com essa abstração, e não com os tipos concretos dos objetos.

Por exemplo, o jogo deve ser capaz de fazer:

```
None
Jogo
```

```
|— Personagem
|— Personagem
|— Efeito
|— ...
```

e, ao passar o turno, chamar `novoTurno()` em cada objeto registrado.

O `Jogo` não deve precisar saber se um objeto é um personagem, efeito, NPC etc.

2. Personagens

Crie uma classe `Personagem`.

Cada personagem deve possuir:

- nome;
- vida atual;
- vida máxima;
- nível;
- experiência;
- uma arma equipada;
- um inventário.

Um personagem deve ser capaz de:

- atacar outro personagem;
- receber dano;
- ser curado;
- verificar se está vivo;
- ganhar experiência;
- subir de nível;
- adicionar itens ao inventário.

A vida e a experiência devem ser **encapsuladas**. Código externo não deve poder simplesmente alterar esses valores.

Ao subir de nível:

- o nível aumenta;

- a vida máxima aumenta;
- a vida é restaurada ao máximo;
- a experiência é reiniciada.

Um personagem também pode precisar reagir à passagem do turno. Nesse caso, ele deve implementar `AtualizavelPorTurno`.

3. Armas

Crie uma interface `Arma`.

O personagem deve possuir uma arma, mas deve depender apenas da interface, e não de uma implementação concreta.

Implemente pelo menos três armas:

Espada

- causa uma quantidade fixa de dano;
- possui cooldown.

Arco

- causa dano;
- possui uma quantidade limitada de flechas;
- cada ataque consome uma flecha;
- pode recarregar flechas;
- possui cooldown.

Varinha mágica

- causa dano;
- consome mana a cada ataque;
- não pode atacar sem mana suficiente;
- pode recuperar mana;
- possui cooldown.

As três armas devem ter comportamentos diferentes.

O código do personagem, entretanto, deve ser capaz de trabalhar com qualquer uma delas através da interface `Arma`.

Não utilize `instanceof` para descobrir qual arma está equipada.

4. Cooldown

Crie uma classe `Cooldown` para encapsular a lógica de cooldown.

O cooldown deve:

- possuir uma duração em turnos;
- informar se está disponível;
- ser iniciado quando a arma é utilizada;
- diminuir a cada novo turno.

Por exemplo, uma arma com cooldown de `2` utilizada no turno atual deve ficar indisponível pelos dois turnos seguintes.

As armas devem **compor** um `Cooldown`, em vez de cada uma implementar sua própria lógica de contagem de turnos.

Considere:

```
None
Arco
└─ Cooldown
```

O `Jogo` não deve conhecer esse detalhe interno.

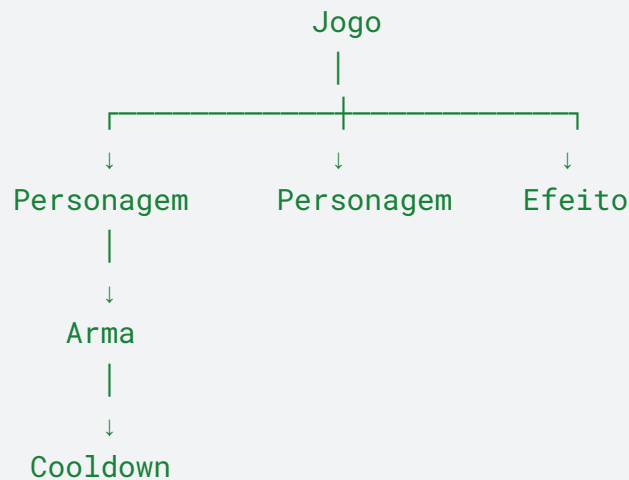
5. Passagem de turnos e Observer

O `Jogo` é responsável por controlar a passagem do tempo do jogo.

Quando um novo turno começa, ele deve notificar os objetos registrados como `AtualizavelPorTurno`.

Por exemplo:

None



Observe que o **Jogo** não deve acessar diretamente o **Cooldown** de uma arma.

O exercício deve explorar a ideia de **Observer**: o jogo funciona como o objeto que produz o evento "novo turno", enquanto os objetos interessados recebem essa notificação.

Não é necessário implementar um Observer genérico; a interface **AtualizavelPorTurno** já é suficiente para representar essa ideia.

6. Inventário

Cada personagem possui um **Inventario**.

O inventário deve ser **criado pelo próprio personagem**, caracterizando uma relação de composição.

Ele deve permitir:

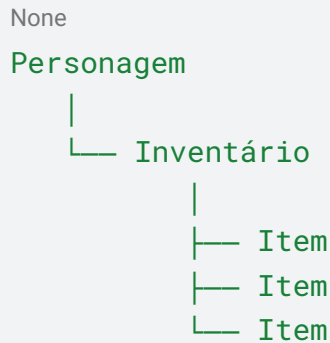
- adicionar itens;
- remover itens;
- listar itens.

Um **Item** deve possuir pelo menos:

- nome;
- valor.

O inventário deve manter o controle de sua própria coleção. Código externo não deve poder modificar diretamente o array interno de itens.

A estrutura deve representar uma composição:



7. Associação × composição

O projeto deve conter exemplos claros dos dois conceitos.

Associação

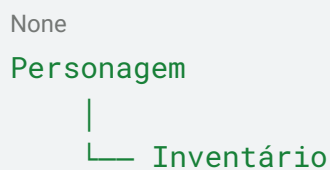
A arma existe independentemente do personagem:



Uma espada pode ser criada e posteriormente equipada por um personagem.

Composição

O inventário é criado e mantido pelo personagem:



Além disso, armas podem utilizar composição para incorporar comportamentos:

None

Arco

|

Cooldown

8. Efeitos temporários

Adicione efeitos que possam permanecer ativos durante vários turnos.

Crie uma abstração para efeitos, por exemplo:

None

Efeito

Implemente pelo menos dois:

Veneno

- causa dano ao personagem a cada turno;
- permanece ativo durante uma determinada quantidade de turnos.

Regeneração

- recupera vida a cada turno;
- também possui uma duração.

Os efeitos devem poder ser aplicados a personagens e devem reagir à passagem dos turnos.

Mais de um efeito pode estar ativo simultaneamente.

9. Polimorfismo

O sistema deve utilizar polimorfismo em diferentes pontos.

Por exemplo:

None

Arma

└─ Espada

└─ Arco

└─ VarinhaMagica

e:

None

AtualizavelPorTurno

└─ Personagem

└─ Efeito

└─ ...

O código que utiliza essas abstrações deve depender dos **contratos**, e não das classes concretas.

Evite condicionais que verificam explicitamente o tipo dos objetos.

10. Cenário de execução

Ao final, crie uma pequena simulação de uma partida.

O cenário deve demonstrar, pelo menos:

1. Criação de um **Jogo**.
2. Criação de personagens.
3. Criação de diferentes armas.
4. Equipamento das armas pelos personagens.
5. Adição dos personagens ao jogo.
6. Ataques entre personagens.
7. Funcionamento de cooldown.
8. Consumo e recarga de flechas.
9. Consumo e recuperação de mana.
10. Criação e utilização do inventário.
11. Aplicação de efeitos temporários.
12. Passagem de vários turnos.
13. Atualização automática dos objetos interessados quando o turno passa.

14. Um personagem recebendo dano e sendo curado.
 15. Um personagem ganhando experiência e subindo de nível.
-

Restrições de projeto

Durante a implementação:

- utilize **interfaces para definir contratos**;
- utilize **composição para reutilizar comportamentos**;
- procure manter os atributos encapsulados;
- não exponha diretamente coleções mutáveis;
- evite duplicação de lógica;
- **Personagem** deve depender da abstração **Arma**;
- **Jogo** deve depender da abstração **AtualizavelPorTurno**;
- **Jogo** não deve conhecer os detalhes internos dos objetos que atualiza;
- evite **instanceof** para controlar comportamentos;
- cada classe deve ser responsável por suas próprias regras.

Objetivo

O objetivo não é apenas fazer o RPG funcionar. O principal objetivo é praticar como **modelar um sistema orientado a objetos no qual objetos colaboram através de abstrações**, mantendo baixo acoplamento e responsabilidades bem definidas.

Ao final, você deverá conseguir identificar no projeto exemplos de:

classe → **encapsulamento** → **associação** → **composição** → **interface** → **polimorfismo** → **Observer/eventos**.